Phase 1 — Technical Documentation

ESL Orchestrator

23.03.2026

João Pires - Devoteam

Sonae MC

Contents

1. Introduction

1.1. Purpose

This document provides the technical documentation for Phase I of the ESL (Electronic Shelf Label) Orchestrator platform at Sonae MC. The platform synchronises ESL data — stores, products, labels and access points — from the Vusion ecosystem into a centralised PostgreSQL database and exposes it through a read-only REST API.

Phase I delivers three components:

Component	Role
Data Pipeline	Scheduled job that fetches data from Vusion Manager and VLink APIs, transforms it, and writes it to PostgreSQL
Database	Flyway-managed PostgreSQL schema that stores all ESL entities and synchronisation state
DataFetch API	REST API that serves the stored data to downstream consumers with pagination and search

1.2. Scope

This document covers the architecture, design, deployment, and operational aspects of the Phase I components. Future phases will introduce additional components and capabilities, which will be documented separately.

1.3. Audience

This document is intended for:

- Technical stakeholders who need to understand the system's architecture and design decisions
- Developers who will extend or maintain the platform
- Operations teams who will deploy, configure, and monitor the system
- Project managers and product owners who need visibility into the platform's capabilities

Technical sections include code and configuration examples with explanations. Architecture diagrams provide a visual overview suitable for all readers.

1.4. How to Read This Document

The document is structured in the following order:

1. System Architecture — High-level view of all components and how they interact
2. Database — Schema design, tables, and migration management
3. Data Pipeline — Synchronisation engine, connectors, and configuration
4. DataFetch API — REST API, endpoints, search, and security
5. Deployment & Infrastructure — Kubernetes, ArgoCD, secrets, and networking
6. Operations & Maintenance — Monitoring, common tasks, and troubleshooting

Readers looking for a high-level understanding can focus on the Architecture section. Those responsible for day-to-day operations should pay particular attention to the Deployment and Operations sections.

1.5. Glossary

Term	Description
ESL	Electronic Shelf Label — digital price tags used in retail stores
Vusion	SES-imagotag's ESL platform, consisting of hardware and cloud services
Vusion Manager	Vusion's store management API — provides store metadata
VLink	Vusion's in-store API — provides product, label, and access point data
Access Point	Radio transmitter hardware in a store that communicates with ESL labels
ArgoCD	GitOps continuous delivery tool for Kubernetes
Flyway	Database migration tool that versions schema changes as SQL files
Helm	Kubernetes package manager used to template deployment manifests
CronJob	Kubernetes resource that runs a job on a recurring schedule
UPSERT	Database operation that inserts a row or updates it if it already exists
Incremental Sync	Synchronisation mode that only fetches records modified since the last run
Full Sync	Synchronisation mode that fetches all records regardless of modification date
External Secrets Operator	Kubernetes operator that syncs secrets from external vaults
OpenTelemetry	Observability framework for distributed tracing and metrics
Prometheus	Monitoring system used for metrics collection and alerting

2. System Architecture

2.1. Overview

The ESL Orchestrator is a data integration platform that bridges the Vusion ESL ecosystem with Sonae MC's internal systems. It follows a three-stage architecture: ingest, store, and serve.

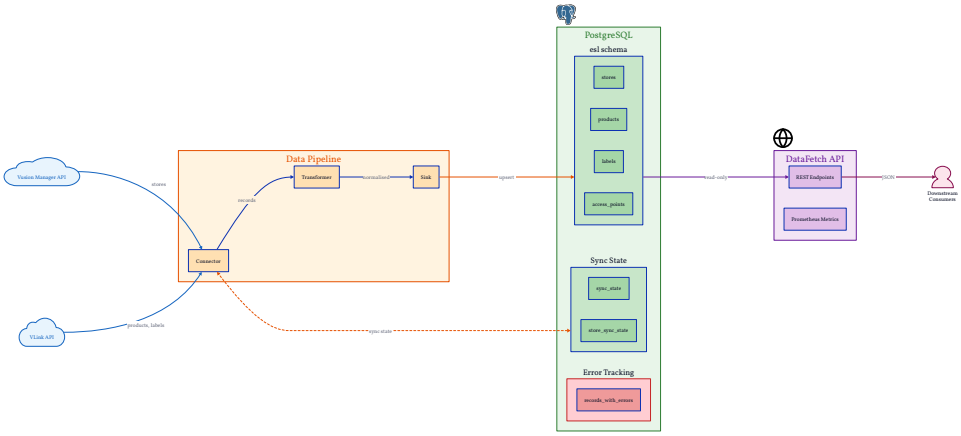

Figure 1: System Architecture

External ESL data flows through the platform in one direction:

1. The Data Pipeline connects to the Vusion Manager and VLink APIs to fetch store, product, label, and access point data
2. Fetched data is transformed and written to the PostgreSQL database using upsert semantics
3. The DataFetch API exposes the stored data to downstream consumers through a read-only REST interface

The pipeline runs as a Kubernetes CronJob on a configurable schedule. The API runs as a continuously available Deployment behind a Traefik ingress.

2.2. Data Flow

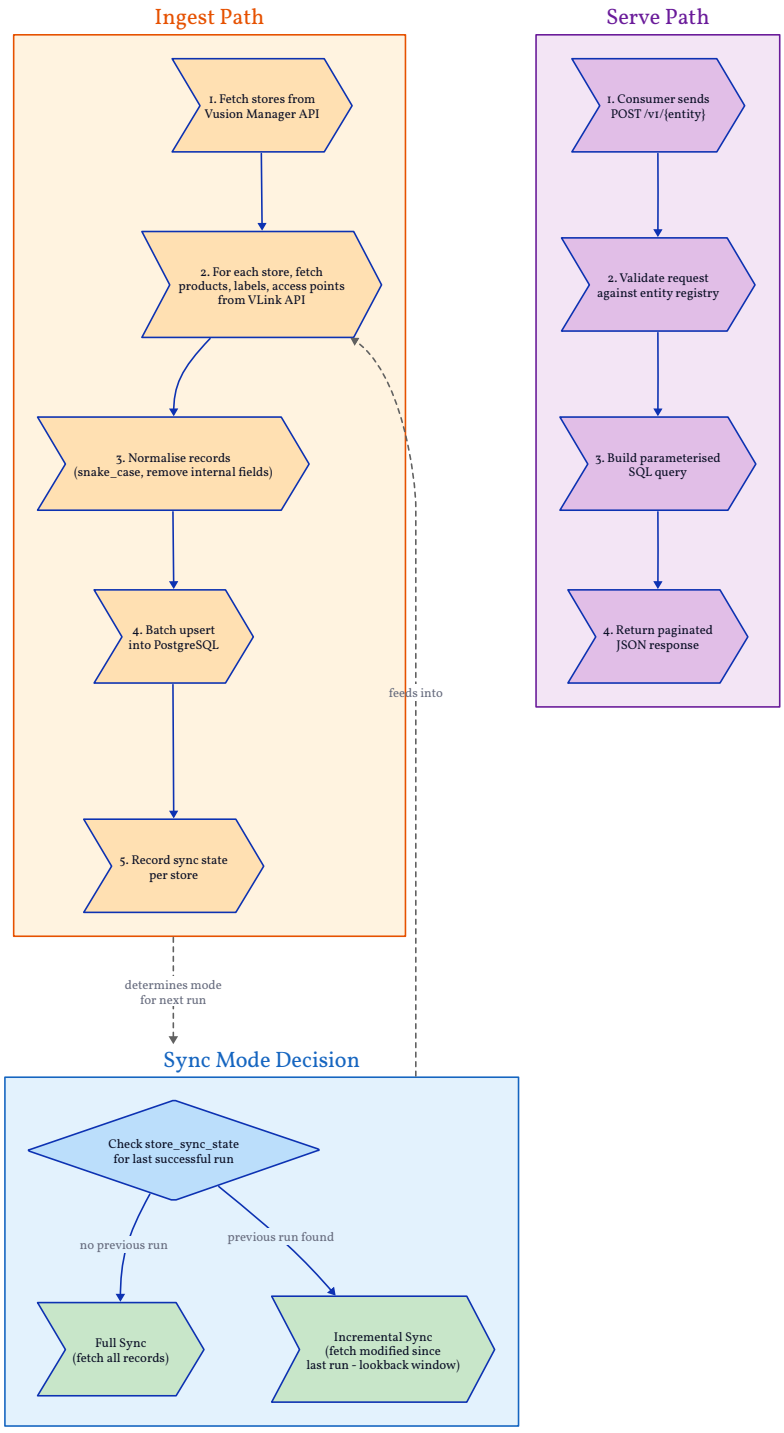

Figure 2: Data Flow

The data flow follows two distinct paths:

Ingest path (Data Pipeline → Database):

1. The pipeline queries the Vusion Manager API to retrieve the list of stores
2. For each store, it queries the VLink API for products, labels, and access points
3. Records are normalised (field names converted to snake_case, internal fields removed)
4. Normalised records are written to PostgreSQL in batches using `INSERT ... ON CONFLICT DO UPDATE`
5. Synchronisation state is recorded per store to enable incremental sync on subsequent runs

Serve path (Database → DataFetch API → Consumers):

1. Consumers send POST requests to the DataFetch API specifying the entity, pagination, and optional search filters
2. The API validates the request against the entity registry, builds a parameterised SQL query, and returns the results as JSON

2.3. Component Responsibilities

Component	Type	Technology	Responsibility
Data Pipeline	CronJob	Go, Cobra, pgx	Fetch data from Vusion APIs, transform, and write to database
Database	Job (PreSync)	PostgreSQL 18, Flyway 12	Schema management, data storage, sync state tracking
DataFetch API	Deployment	Go, Gin, pgx	Read-only REST API with pagination, search, and metrics

2.4. Environment Overview

The platform is deployed across three environments, each in its own Kubernetes namespace:

Environment	Namespace	Purpose
Development	`instore-esl-orchestrator-dev`	Feature development and early testing
Preproduction	`instore-esl-orchestrator-pp`	Integration testing with production-like configuration
Production	`instore-esl-orchestrator-prd`	Live operations serving real store data

All environments follow the same deployment topology managed by ArgoCD. Configuration differences (schedules, store filters, resource limits, hostnames) are controlled through environment-specific Helm values files.

2.5. Technology Stack

2.5.1. Languages and Frameworks

Technology	Version	Used By
Go	1.25+	Data Pipeline, DataFetch API
PostgreSQL	18	Database
Flyway	12	Database migrations

2.5.2. Key Libraries

Library	Purpose
pgx/v5	PostgreSQL driver with native protocol support and connection pooling
Cobra	CLI framework for the Data Pipeline
Gin	HTTP framework for the DataFetch API
Zap	Structured logging across all Go components
OpenTelemetry	Distributed tracing in the Data Pipeline
Prometheus client	Metrics instrumentation in the DataFetch API

2.5.3. Infrastructure

Technology	Purpose
Kubernetes	Container orchestration
Helm	Deployment templating
ArgoCD	GitOps continuous delivery
Traefik	Ingress controller and TLS termination
External Secrets Operator	Vault-to-Kubernetes secret synchronisation
HashiCorp Vault	Secret storage (on-premise environments)

3. Database

3.1. Purpose

The database component manages the PostgreSQL schema that underpins the entire ESL Orchestrator platform. It stores all entity data synchronised from the Vusion ecosystem — stores, products, labels, and access points — as well as the synchronisation state used by the Data Pipeline to determine whether each store needs a full or incremental sync.

All schema changes are version-controlled using Flyway, ensuring reproducible and auditable migrations across environments.

3.2. Technology Stack

Technology	Version	Purpose
PostgreSQL	18	Relational database engine
Flyway	12	Versioned SQL migration framework
pg_trgm	(extension)	Trigram-based text search for fuzzy matching

3.3. Schema Overview

All tables live in the `esl` schema. The schema contains seven tables organised into three groups:

Entity data — synchronised from the Vusion APIs:

- `stores` — physical store metadata

- `products` — product catalogue per store
- `labels` — ESL label status, hardware, and transmission data
- `access_points` — radio transmitter hardware per store

Synchronisation state — used by the Data Pipeline to track sync history:

- `sync_state` — one row per pipeline run (aggregate metrics)
- `store_sync_state` — one row per store per run (enables per-store incremental sync)

Error tracking:

- `records_with_errors` — failed records captured for debugging and retry

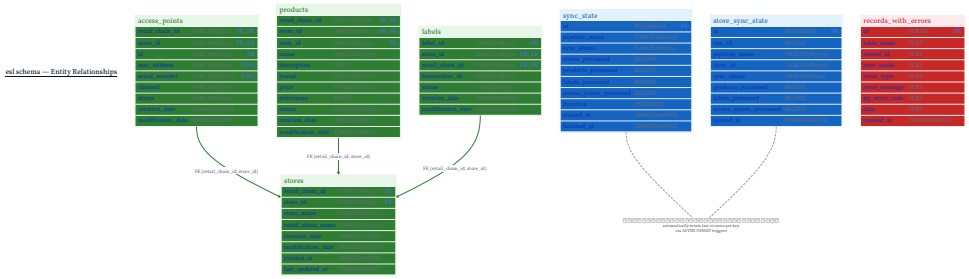

Figure 3: ER Diagram

3.4. Table Details

3.4.1. stores

The root entity. All other entity tables reference stores via a composite foreign key.

Column	Type	Description
retail_chain_id	VARCHAR(50)	Retail chain identifier (PK)
store_id	VARCHAR(50)	Store identifier (PK)
store_name	VARCHAR(100)	Human-readable store name
retail_chain_name	VARCHAR(100)	Retail chain name
software_setting_file_name	VARCHAR(255)	Vusion software configuration file
software_setting_version	VARCHAR(50)	Software version
software_setting_last_update	TIMESTAMPTZ	Last software update
creation_date	TIMESTAMPTZ	Created in Vusion
modification_date	TIMESTAMPTZ	Last modified in Vusion
created_at	TIMESTAMPTZ	Row insertion timestamp
last_updated_at	TIMESTAMPTZ	Last upsert timestamp

Primary key: `(retail_chain_id, store_id)`

Indexes: B-tree on `store_id`; GIN trigram on `store_name` and `retail_chain_name` for text search.

3.4.2. access_points

Radio transmitter hardware installed in each store. Each access point communicates with nearby ESL labels.

Column	Type	Description
retail_chain_id	VARCHAR(50)	Retail chain identifier (PK, FK)
store_id	VARCHAR(50)	Store identifier (PK, FK)
id	VARCHAR(50)	Access point identifier (PK)
mac_address	VARCHAR(50)	Hardware MAC address (unique)
serial_number	VARCHAR(100)	Hardware serial number (unique)
channel	VARCHAR(10)	Radio channel
status	VARCHAR(50)	Operational status
connectivity_status	VARCHAR(50)	Online/offline status
connectivity_last_online_date	TIMESTAMPTZ	Last seen online
connectivity_last_offline_date	TIMESTAMPTZ	Last went offline
creation_date	TIMESTAMPTZ	Created in Vusion
modification_date	TIMESTAMPTZ	Last modified in Vusion
created_at	TIMESTAMPTZ	Row insertion timestamp
last_updated_at	TIMESTAMPTZ	Last upsert timestamp

Primary key: `(retail_chain_id, store_id, id)`
Foreign key: `(retail_chain_id, store_id)` → `stores`

Indexes: B-tree on `store_id` and `id`; unique indexes on `mac_address` and `serial_number`.

3.4.3. products

Product catalogue data linked to each store. Includes core product attributes and a set of custom fields used by Sonae MC's retail operations.

Column	Type	Description
retail_chain_id	VARCHAR(50)	Retail chain identifier (PK, FK)
store_id	VARCHAR(50)	Store identifier (PK, FK)
item_id	VARCHAR(50)	Product item identifier (PK)
id	VARCHAR(300)	Vusion product identifier
name	VARCHAR(255)	Product name
description	TEXT	Product description
brand	VARCHAR(100)	Brand
price	NUMERIC(12,2)	Current price
references	TEXT[]	Reference codes (array)
status	VARCHAR(300)	Product status
matching_count	INTEGER	Number of label matches
matching_matched	BOOLEAN	Whether a label is matched
matching_labels	TEXT[]	Matched label IDs (array)
custom_*	various	30+ custom fields for retail operations
creation_date	TIMESTAMPTZ	Created in Vusion
modification_date	TIMESTAMPTZ	Last modified in Vusion
deletion_date	TIMESTAMPTZ	Soft deletion date
created_at	TIMESTAMPTZ	Row insertion timestamp
last_updated_at	TIMESTAMPTZ	Last upsert timestamp

Primary key: (retail_chain_id, store_id, item_id)
Foreign key: (retail_chain_id, store_id) → stores

Indexes: B-tree on item_id, status, creation_date, modification_date, custom_dept, custom_class, custom_subclass, custom_ticket_subtype, custom_status; GIN on description, name (trigram), references, matching_labels (array).

3.4.4. labels

ESL label data including hardware details, transmission history, product matching, and connectivity status.

Column	Type	Description
label_id	VARCHAR(50)	Label identifier (PK)
store_id	VARCHAR(50)	Store identifier (PK, FK)
retail_chain_id	VARCHAR(50)	Retail chain identifier (PK, FK)
transmitter_id	VARCHAR(50)	Current access point
previous_transmitter_id	VARCHAR(50)	Previous access point
status	VARCHAR(50)	Label status
hardware_type_name	VARCHAR(100)	Hardware model
hardware_battery	VARCHAR(50)	Battery level
package_*	various	Package registration data
transmission_*	various	Transmission dates and history
matching_*	various	Product matching details
connectivity_*	various	Connectivity status and signal
billing_activate_date	TIMESTAMPTZ	Billing activation date
last_join_timestamp	TIMESTAMPTZ	Last network join
creation_date	TIMESTAMPTZ	Created in Vusion
modification_date	TIMESTAMPTZ	Last modified in Vusion
created_at	TIMESTAMPTZ	Row insertion timestamp
last_updated_at	TIMESTAMPTZ	Last upsert timestamp

Primary key: `(store_id, retail_chain_id, label_id)`
Foreign key: `(retail_chain_id, store_id)` → `stores`

Indexes: B-tree on `transmitter_id`, `matching_items_item_id`, `package_package_id`, `status`, `hardware_type_name`, `hardware_battery`, `matching_scenario_scenario_id`, `matching_items_modification_date`; GIN on `matching_items_references` (array).

3.4.5. sync_state

Tracks aggregate metrics per pipeline execution. Used for operational monitoring and historical analysis.

Column	Type	Description
id	BIGSERIAL	Auto-increment identifier (PK)
pipeline_name	VARCHAR(255)	Pipeline identifier
sync_status	VARCHAR(20)	success / failed
stores_processed	BIGINT	Number of stores processed
products_processed	BIGINT	Number of products processed
labels_processed	BIGINT	Number of labels processed
access_points_processed	BIGINT	Number of access points processed
duration	INTERVAL	Total run duration
started_at	TIMESTAMPTZ	Run start time
finished_at	TIMESTAMPTZ	Run end time
error_message	TEXT	Error details (if failed)

Retention trigger: Automatically keeps only the last 20 rows per `pipeline_name`.

3.4.6. store_sync_state

Tracks per-store sync history. The Data Pipeline queries this table to decide whether each store needs a full sync (no previous successful run) or incremental sync (fetch only records modified since the last run).

Column	Type	Description
id	BIGSERIAL	Auto-increment identifier (PK)
run_id	BIGINT	Reference to the parent sync_state run
pipeline_name	VARCHAR(255)	Pipeline identifier
store_id	VARCHAR(255)	Store identifier
sync_status	VARCHAR(20)	success / failed / cancelled
products_processed	BIGINT	Products synced for this store
labels_processed	BIGINT	Labels synced for this store
access_points_processed	BIGINT	Access points synced for this store
synced_at	TIMESTAMPTZ	Sync completion time
error_message	TEXT	Error details (if failed)

Retention trigger: Automatically keeps only the last 20 rows per (pipeline_name, store_id).

3.4.7. records_with_errors

Captures individual records that failed during sync for debugging and potential retry.

Column	Type	Description
id	SERIAL	Auto-increment identifier (PK)
table_name	TEXT	Target table that rejected the record
record_id	TEXT	Identifier of the failed record
sync_mode	TEXT	full or incremental
error_type	TEXT	Error classification
error_message	TEXT	Error description
detail	TEXT	Detailed error context
pg_error_code	TEXT	PostgreSQL error code
data	TEXT	Raw record payload
created_at	TIMESTAMPTZ	Capture timestamp

3.5. Key Design Decisions

3.5.1. Composite primary keys

All entity tables use composite primary keys that include `retail_chain_id` and `store_id`. This supports multi-tenant isolation at the row level — the same store number can exist in different retail chains without conflict.

3.5.2. Upsert strategy

The schema is designed for `INSERT ... ON CONFLICT DO UPDATE` operations. The Data Pipeline writes data using upsert semantics, meaning records are inserted on first sync and updated on subsequent syncs. This makes the pipeline idempotent — re-running it produces the same result.

3.5.3. Trigram text search

The `pg_trgm` extension enables GIN trigram indexes on text fields like `store_name`, `name`, and `description`. This allows efficient fuzzy text search through the DataFetch API without a dedicated search engine.

3.5.4. Retention triggers

The `sync_state` and `store_sync_state` tables use `AFTER INSERT` triggers that automatically delete rows beyond the most recent 20 per key. This prevents unbounded growth of operational metadata while retaining enough history for debugging.

3.5.5. PgBouncer compatibility

Migration `V1.0.0.13` sets `extra_float_digits` and `statement_timeout` at the database role level. This is necessary because PgBouncer strips unknown startup parameters from client connections, but the PostgreSQL JDBC driver (used by Flyway) sends these parameters at connect time. Without this workaround, Flyway connections through PgBouncer would fail.

3.6. Migration Management

3.6.1. Framework

Migrations are managed by Flyway and stored as versioned SQL files in the `sql/` directory.

3.6.2. File naming convention

```
V{major}.{minor}.{patch}.{seq}__{description}.sql
```

- Double underscore (__) separates the version from the description
- Sequential numbering (`01`, `02`, …) determines execution order
- Descriptions use `snake_case`
- Tables and their indexes are defined in separate migrations

3.6.3. Current migrations

Version	Description
V1.0.0.01	Create `esl` schema
V1.0.0.02	Enable `pg_trgm` extension
V1.0.0.03	Create `stores` table
V1.0.0.04	Create indexes on `stores`
V1.0.0.05	Create `access_points` table
V1.0.0.06	Create indexes on `access_points`
V1.0.0.07	Create `products` table
V1.0.0.08	Create indexes on `products`
V1.0.0.09	Create `labels` table
V1.0.0.10	Create indexes on `labels`
V1.0.0.11	Create `records_with_errors` table
V1.0.0.12	Create `sync_state` table with retention trigger
V1.0.0.13	Configure PgBouncer compatibility
V1.0.0.14	Create `store_sync_state` table with retention trigger

3.6.4. Rules

- Never modify an already-applied migration — Flyway validates checksums and will reject changes
- Always create new migrations for schema changes
- Flyway runs automatically as a Kubernetes PreSync hook before the Data Pipeline and DataFetch API are deployed

4. Data Pipeline

4.1. Purpose

The Data Pipeline is the ingestion engine of the ESL Orchestrator. It connects to the Vusion ecosystem APIs, fetches store and entity data, normalises it, and writes

it to PostgreSQL using upsert semantics. It runs as a Kubernetes CronJob on a configurable schedule.

The pipeline supports incremental synchronisation — on subsequent runs, it only fetches records modified since the last successful sync for each store, reducing API calls by 60–98% compared to a full sync.

4.2. Technology Stack

Technology	Version	Purpose
Go	1.25+	Application language
Cobra	—	CLI framework
pgx/v5	—	PostgreSQL driver with connection pooling
Zap	—	Structured logging
OpenTelemetry	—	Distributed tracing and metrics
Viper	—	Configuration management with env var overrides

4.3. Architecture

The pipeline follows a Connector → Transformer → Sink pattern:

Pipeline Architecture

Figure 4: Pipeline Architecture

1. Connector — Fetches data from Vusion Manager and VLink APIs
2. Transformer — Normalises records (field extraction, snake_case conversion)
3. Sink — Writes normalised records to PostgreSQL in batches

Records flow through buffered channels, processed by a configurable number of concurrent workers.

4.4. CLI

The pipeline ships as a single binary (`eslorchestrator`) with three subcommands:

4.4.1. run

Executes the pipeline using the provided configuration file.

```
eslorchestrator run config.yaml
eslorchestrator run -c config.yaml --log-level debug
eslorchestrator run config.yaml -v   # verbose (sets log-level to debug)
```

Flag	Default	Description
`-c, --config`	`$HOME/.eslorchestrator.yaml`	Config file path
`-l, --log-level`	`info`	Logging level: debug, info, warn, error
`-v, --verbose`	`false`	Enable verbose output

Each invocation generates a unique execution ID (UUID) for tracing across logs.

4.4.2. validate

Validates configuration syntax and structure without executing the pipeline.

```
eslorchestrator validate config.yaml
```

Outputs pipeline metadata on success: name, transform type, sink type, worker count, buffer size, and telemetry status.

4.4.3. version

Displays build information: version, commit hash, build date, Go version, and platform.

```
eslorchestrator version
```

4.5. Connector

The Vusion connector implements a two-phase synchronisation strategy:

4.5.1. Phase 1 — Store Discovery

1. Fetch all stores from the Vusion Manager API (paginated, page size: 100)
2. Emit store records to the pipeline output channel
3. Buffer stores in memory for Phase 2
4. Apply store filters:
   - `filter_stores` — explicit list of store IDs (takes priority)
   - `filter_retail_chains` — retail chain IDs (used only if `filter_stores` is empty)
   - Both empty — process all stores

4.5.2. Phase 2 — Per-Store Entity Fetching

1. Load the latest successful sync state per store from the database
2. Launch a worker pool (`store_concurrency`, default: 10) to process stores in parallel
3. For each store, determine sync mode:
   - Full sync — no prior successful run exists → fetch all records

- Incremental sync — prior run exists → fetch records modified since `synced_at - lookback_window`

4. Fetch products, labels, and access points from VLink API

4.5.3. External APIs

Vusion Manager — store metadata:

- Endpoint: `/stores/search`
- Returns: store ID, retail chain, name, software settings, timestamps
- Auth: `Ocp-Apim-Subscription-Key` and `ApiKey` headers

VLink — per-store entity data:

- Endpoints:
  - `/stores/{storeID}/products` (full sync)
  - `/stores/{storeID}/products/modified-since?since={timestamp}` (incremental)
  - `/stores/{storeID}/labels` and `/stores/{storeID}/labels/modified-since?since={timestamp}`
- Page size: 1000
- Auth: same headers as Vusion Manager

Access points are extracted directly from the store object (`transmissionSystems.highFrequency.transmitters`), not fetched via a separate API call.

4.5.4. Error Handling

- Phase 1 failure (store fetch) — pipeline aborts immediately
- Phase 2 failure (single store) — other stores continue processing; failure recorded in state

4.6. Transformer

The normaliser transformer extracts fields from nested API responses and flattens them into database-ready column names.

4.6.1. Transformation Steps

1. Extract values using dot-notation paths (e.g., `software.setting.fileName`)
2. Convert field names to snake_case (`fileName` → `file_name`)
3. Flatten nested paths (`software.setting.fileName` → `software_setting_file_name`)
4. Remove internal fields (e.g., `_score`)

4.6.2. Entity Field Mappings

Each entity type has a defined set of fields to extract:

Store — 9 fields: `retail_chain_id`, `store_id`, `store_name`, `retail_chain_name`, `software_setting_file_name`, `software_setting_version`, `software_setting_last_update`, `creation_date`, `modification_date`

Product — 45+ fields: core attributes (`item_id`, `name`, `description`, `brand`, `price`, `status`), matching data (`matching_count`, `matching_matched`, `matching_labels`), and 30+ custom fields (`custom_dept`, `custom_class`, `custom_status`, etc.)

Label — 30+ fields: hardware info (`hardware_type_name`, `hardware_battery`), transmission history (`transmission_registration_date`, `transmission_last_successful_transmission_date`), matching details, and connectivity status

Access Point — 12 fields: identifiers (`id`, `mac_address`, `serial_number`), status, channel, and connectivity data

4.7. Sink

The PostgreSQL sink writes normalised records using batched upsert operations.

4.7.1. Batching

Records are queued to a background worker that flushes a batch when any of these conditions is met:

Condition	Default	Config Key
Batch reaches size limit	100 records	`sink.postgres.batch_size`
Time since first record in batch	1s	`sink.postgres.batch_timeout`
No new records (idle)	100ms	`sink.postgres.idle_flush_timeout`

4.7.2. Upsert Logic

Each batch is executed as a single transaction using `pgx.Batch`:

```
INSERT INTO {table} ({columns})
VALUES ($1, $2, ...)
ON CONFLICT ({conflict_keys})
DO UPDATE SET {column} = EXCLUDED.{column}, ...
```

Conflict keys are configured per entity type (e.g., `item_id, store_id, retail_chain_id` for products).

4.7.3. Error Handling and Retry

Failed records are classified as permanent or transient:

Type	PG Error Codes	Action
Permanent	23505 (unique violation), 23502 (not null), 42601 (syntax)	Stored in `records_with_errors`, not retried
Transient	40P01/40001 (deadlock), 57P01–57P03 (system)	Retried with exponential backoff

Retry configuration:

Setting	Default	Config Key
Max retries	3	`sink.postgres.max_retries`
Initial delay	1s	`sink.postgres.retry_delay`
Backoff multiplier	2.0	`sink.postgres.retry_backoff`
Max delay	30s	`sink.postgres.max_retry_delay`

4.7.4. Connection Pool

Setting	Default	Config Key
Max connections	10	`sink.postgres.max_connections`
Min connections	2	`sink.postgres.min_connections`
Max lifetime	1h	`sink.postgres.max_conn_lifetime`
Idle timeout	30m	`sink.postgres.max_conn_idle_time`
Health check	1m	`sink.postgres.health_check_period`

4.8. State Management

The pipeline tracks synchronisation history in two database tables to enable per-store incremental sync decisions.

4.8.1. sync_state

One row per pipeline run. Records aggregate metrics: total stores/products/labels/access points processed, duration, status, and error message.

4.8.2. store_sync_state

One row per store per run. Records per-store metrics and the `synced_at` timestamp — the pipeline start time (not finish time) — which is used as the incremental sync cutoff on the next run.

4.8.3. Sync Mode Decision

For each store in a run:

1. Query `store_sync_state` for the most recent successful run
2. If no successful run exists → full sync
3. If a successful run exists → incremental sync with cutoff = `synced_at - lookback_window`

The `lookback_window` (e.g., `5m`) provides a safety margin against clock skew between systems.

4.9. Configuration Reference

The pipeline is configured via a YAML file with environment variable overrides. Env vars use uppercase dot-to-underscore notation (e.g., `CONNECTOR_TIMEOUT`).

4.9.1. Pipeline

```yaml
pipeline:
  name: "esl-orchestrator"          # Required — pipeline identifier
  description: "ESL sync pipeline"  # Optional
  performance:
    worker_count: 10                # Concurrent transform workers
    buffer_size: 1000               # Channel buffer capacity
```

4.9.2. Connector

```yaml
connector:
  vusion_manager:
    base_url: "https://..."         # Required
    subscription_key: "..."         # Required
    api_key: "..."                  # Required
  vlink:
    base_url: "https://..."         # Required
    subscription_key: "..."         # Required
    api_key: "..."                  # Required
  timeout: "30s"                    # HTTP client timeout
  buffer_size: 500                  # Record stream buffer
  store_concurrency: 10             # Parallel store processing
  filter_stores: []                 # Explicit store IDs (takes priority)
```

```
  filter_retail_chains: []          # Retail chain IDs (fallback)
  sync:
    lookback_window: "5m"          # Safety margin for incremental cutoff
    state_store:
      type: "postgres"
      postgres:
        host: "localhost"
        port: 5432
        database: "eslorchestrator"
        username: "..."
        password: "..."
        schema: "esl"
        ssl_mode: "disable"
```

4.9.3. Transform

```
transform:
  type: "normalizer"               # "normalizer" or "passthrough"
```

4.9.4. Sink

```
sink:
  type: "postgres"
  postgres:
    host: "localhost"
    port: 5432
    database: "eslorchestrator"
    username: "..."
    password: "..."
    schema: "esl"
    ssl_mode: "disable"
    batch_size: 500
    batch_timeout: "5s"
    max_retries: 3
    max_connections: 10
    tables:
      store:
        name: "stores"
        conflict_keys: ["store_id", "retail_chain_id"]
      product:
        name: "products"
        conflict_keys: ["item_id", "store_id", "retail_chain_id"]
      label:
        name: "labels"
        conflict_keys: ["label_id", "store_id", "retail_chain_id"]
```

```
    accesspoint:
      name: "access_points"
      conflict_keys: ["id", "store_id", "retail_chain_id"]
```

4.9.5. Telemetry

```
telemetry:
  enabled: true
  otlp_endpoint: "localhost:4317"
  service_name: "eslorchestrator"
  service_version: "1.0.0"
  environment: "production"
```

4.10. Observability

4.10.1. Structured Logging

All log entries include the execution ID, pipeline name, and contextual fields (store ID, entity type, record count). Log level is configurable via `--log-level` flag or `LOG_LEVEL` environment variable.

4.10.2. Distributed Tracing

When telemetry is enabled, the pipeline emits OpenTelemetry spans via OTLP/gRPC:

Span	Scope
`pipeline.run`	Overall pipeline execution
`connector.vusion.read`	Vusion API data fetch
`pipeline.process_record`	Individual record processing
`sink.postgres.write`	Record queuing to sink
`sink.postgres.write_batch`	Batch execution

4.10.3. Metrics

Metric	Type	Description
`pipeline.records.processed`	Counter	Successfully processed records
`pipeline.records.failed`	Counter	Failed records
`connector.records.read`	Counter	Records fetched from source
`sink.records.written`	Counter	Records written to database
`pipeline.run.duration`	Histogram	Total run duration (ms)
`connector.latency`	Histogram	API fetch latency (ms)
`sink.latency`	Histogram	Write latency (ms)

All metrics are prefixed with `eslorchestrator.` and include attributes for pipeline name, entity type, and error classification.

5. DataFetch API

5.1. Purpose

The DataFetch API is a read-only REST service that exposes ESL data stored in PostgreSQL to downstream consumers. It provides paginated access to stores, products, labels, and access points with field-level search capabilities.

The API is designed for secure, high-performance data retrieval — all queries use parameterised SQL, input is strictly validated, and the service includes built-in observability via Prometheus metrics and structured logging.

5.2. Technology Stack

Technology	Version	Purpose
Go	1.26+	Application language
Gin	1.11	HTTP framework
pgx/v5	—	PostgreSQL driver with connection pooling
Zap	1.27	Structured logging
Prometheus client	1.23	Metrics instrumentation
Swagger UI	—	Embedded interactive API documentation

5.3. API Endpoints

Method	Path	Description
POST	/v1/{entity_name}	Fetch entity data with pagination and search
GET	/health	Service health check (always 200)
GET	/ready	Readiness check (includes DB connectivity)
GET	/metrics	Prometheus metrics
GET	/openapi.yaml	OpenAPI 3.0 specification
GET	/swagger	Interactive Swagger UI

5.3.1. POST /v1/{entity_name}

The main data retrieval endpoint. Returns rows from the specified entity with optional pagination and search filters.

Path parameter:

- `entity_name` — one of: `stores`, `products`, `labels`, `access_points`

Request body (JSON, all fields optional):

```
{
  "page": 1,
  "pageSize": 10,
  "search": "store_id:bomdia_pt.009648"
}
```

Field	Type	Default	Constraints
page	integer	1	>= 1
pageSize	integer	10	1–100
search	string	(empty)	Max 1000 characters

Response (200 OK):

```
{
  "entityName": "stores",
  "page": 1,
  "pageSize": 10,
  "totalCount": 1523,
```

```
  "rows": [
    {
      "retail_chain_id": "continente_pt",
      "store_id": "001234",
      "store_name": "Continente Matosinhos",
      ...
    }
  ]
}
```

Error responses:

Status	Code	Scenario
400	`INVALID_INPUT`	Bad request body, unknown field in search, invalid search syntax
404	`NOT_FOUND`	Entity name does not exist
413	`INVALID_INPUT`	Request body exceeds 64 KB
500	`DATABASE_ERROR`	Query execution failure
504	`TIMEOUT`	Query exceeded timeout

5.3.2. Health and Readiness

- `GET /health` — returns `{"status": "ok", "service": "datafetch"}` (always 200)
- `GET /ready` — returns `{"status": "ready", "checks": {"database": "ok"}}` or 503 if the database is unreachable (2-second ping timeout)

5.4. Search Syntax

The `search` field supports field-level filtering with three operators:

5.4.1. Equality

`field:value`

Example: `status:active`

5.4.2. OR

`field1:value1 OR field2:value2`

Example: `store_id:bomdia_pt.009648 OR store_id:continente_pt.000001`

5.4.3. AND

`field1:value1 AND field2:value2`

Example: `status:active AND custom_dept:grocery`

> Note: AND and OR cannot be mixed in a single search expression.

5.4.4. Timestamp Range

For timestamp columns only:

`field:[lower_bound TO upper_bound]`

Bounds must be RFC 3339 timestamps or `*` (unbounded). At least one bound is required.

Examples:

- `creation_date:[2024-01-01T00:00:00Z TO 2024-12-31T23:59:59Z]` — closed range
- `creation_date:[* TO 2024-06-01T00:00:00Z]` — open lower bound
- `modification_date:[2024-01-01T00:00:00Z TO *]` — open upper bound

5.4.5. Type Validation

Search values are validated against the column type before query execution:

Column Type	Accepted Values
text	Any string
integer	64-bit signed integer
numeric	Floating-point number
boolean	`true`, `false`, `1`, `0`
timestamp	RFC 3339 format
array	Text value (matched via PostgreSQL `@>` contains operator)

5.4.6. Constraints

- Max search string length: 1000 characters
- Max individual term length: 255 characters
- Max field name length: 64 characters
- Field names: alphanumeric and underscores only

5.5. Entity Registry

Entities are defined in an embedded YAML configuration (`entities.yaml`) that specifies which database tables are queryable, their columns, types, and primary keys. At startup, the registry is validated against the live database schema — if a declared column does not exist in the database, the service will not start.

5.5.1. Available Entities

Entity	Table	Primary Key	Columns (API-visible)
`stores`	`esl.stores`	`retail_chain_id, store_id`	19
`products`	`esl.products`	`retail_chain_id, store_id, item_id`	66
`labels`	`esl.labels`	`store_id, retail_chain_id, label_id`	39
`access_points`	`esl.access_points`	`retail_chain_id, store_id, id`	13

Internal columns (`created_at`, `last_updated_at`) are excluded from API responses and are not searchable.

Results are ordered deterministically by primary key columns.

5.6. Security

5.6.1. SQL Injection Prevention

- All user input is passed as parameterised values (`$1`, `$2`, etc.) — never concatenated into SQL strings
- Entity and column names are sanitised via `pgx.Identifier{}.Sanitize()`
- Field names in search are validated against a strict regex (`^[a-zA-Z0-9_]+$`) and must exist in the entity registry

5.6.2. Input Validation

Input	Limit
Request body size	64 KB
Page size	100 rows
Search string	1000 characters
Individual search term	255 characters
Field name	64 characters
Request ID header	128 characters, alphanumeric + . - _

Unknown JSON fields in the request body are rejected.

5.6.3. Security Headers

All responses include the following headers:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `X-XSS-Protection: 1; mode=block`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Permissions-Policy: geolocation=(), microphone=(), camera=()`
- `Content-Security-Policy` (configured for Swagger UI compatibility)

5.6.4. Container Security

The service runs as a non-root user (UID 1002) in a distroless container with a read-only root filesystem and all Linux capabilities dropped.

5.7. Configuration

The service is configured via environment variables with defaults that vary by environment.

5.7.1. Core Settings

Variable	Dev Default	Prod Default	Description
`APP_ENV`	development	—	Environment profile
`HTTP_PORT`	8080	8080	Server port
`LOG_LEVEL`	info	info	debug, info, warn, error

5.7.2. Database Connection

Variable	Dev Default	Prod Default	Description
DB_HOST	localhost	localhost	PostgreSQL host
DB_PORT	5432	5432	PostgreSQL port
DB_USER	postgres	postgres	Database user
DB_PASSWORD	postgres	postgres	Database password
DB_NAME	datafetch	datafetch	Database name
DB_SCHEMA	public	public	Schema for entity tables
DB_SSL_MODE	disable	require	SSL mode

5.7.3. Connection Pool

Variable	Dev Default	Prod Default	Description
DB_MAX_CONNS	10	25	Maximum connections
DB_MIN_CONNS	2	5	Minimum idle connections
DB_MAX_CONN_LIFETIME	30	60	Connection max lifetime (minutes)
DB_MAX_CONN_IDLE_TIME	15	30	Idle timeout (minutes)
DB_HEALTH_CHECK_PERIOD	30	60	Health check interval (seconds)

5.7.4. Timeouts

Variable	Dev Default	Prod Default	Description
QUERY_TIMEOUT	15	30	Database query timeout (seconds)
HTTP_READ_TIMEOUT	30	30	HTTP read timeout (seconds)
HTTP_WRITE_TIMEOUT	60	60	HTTP write timeout (seconds)

5.8. Observability

5.8.1. Prometheus Metrics

Available at `GET /metrics`.

HTTP metrics:

Metric	Type	Labels	Description
`http_requests_total`	Counter	method, path, status	Total request count
`http_request_duration_seconds`	Histogram	method, path	Request latency
`http_requests_in_flight`	Gauge	—	Current concurrent requests

Database metrics:

Metric	Type	Labels	Description
`database_queries_total`	Counter	table, operation	Total query count
`database_query_duration_seconds`	Histogram	operation	Query latency
`database_connection_pool_size`	Gauge	state	Pool usage (acquired, idle, total)

Connection pool stats are collected every 15 seconds.

5.8.2. Structured Logging

All log entries include `request_id`, `method`, `path`, `status`, and `latency`. Errors include additional context (entity name, search string, error code). Logs are JSON in production and human-readable in development.

5.8.3. Request Tracing

Every request is tagged with a request ID via the `X-Request-ID` header. If not provided by the client, the server generates a UUID v4. The ID is echoed in the response and included in all log entries for correlation.

6. Deployment & Infrastructure

6.1. Overview

The ESL Orchestrator is deployed on Kubernetes using Helm charts and managed by ArgoCD (GitOps). Each environment runs in its own namespace with environment-specific configuration.

The deployment consists of three workload types:

Component	K8s Resource	Purpose
Database Migrations	Job (PreSync)	Flyway schema migrations — must succeed before other components deploy
Data Pipeline	CronJob	Scheduled data synchronisation
DataFetch API	Deployment + Service + Ingress	Continuously running REST API

6.2. ArgoCD Deployment Flow

ArgoCD manages the deployment lifecycle using sync hooks to enforce ordering:

Figure 5: Deployment Flow

6.2.1. PreSync Phase (Weight 0)

The following resources are created first:

- NetworkPolicy — allows the migration job to reach the database
- ServiceAccount — identity for the migration job
- ExternalSecret — pulls Flyway credentials from Vault

These resources are deleted automatically after a successful sync (`HookSucceeded` deletion policy).

6.2.2. PreSync Phase (Weight 1)

- Flyway Migration Job — runs schema migrations against PostgreSQL
- Backoff limit: `0` — if the job fails, the entire sync is aborted
- This prevents deploying application code that depends on a schema change that hasn't been applied

6.2.3. Sync Phase

Once migrations succeed, the remaining resources are deployed:

- DataFetch API: Deployment, Service, Ingress, ExternalSecrets
- Data Pipeline: CronJob, ConfigMap, ExternalSecrets
- NetworkPolicy (main)

6.3. Helm Chart

The chart (`instore-esl-orchestrator`, version `0.1.1`) is templated across environments using values files:

```
helm/
├── Chart.yaml
└── templates/
    ├── datafetch-api.yaml        # Deployment + Service + Ingress
    ├── datapipeline-cronjob.yaml # CronJob + ConfigMap
    ├── dbmigrations.yaml         # PreSync Job + hooks
    └── network-policy.yaml       # NetworkPolicy
```

Each environment has its own values directory:

```
clusters/
├── cluster-dev/
│   ├── values-dev.yaml
│   ├── values-security-dev.yaml
│   └── values-observability-dev.yaml
├── cluster-preproduction/
│   ├── values-pp.yaml
│   ├── values-security-pp.yaml
│   └── values-observability-pp.yaml
└── cluster-production/
    ├── values-prd.yaml
    ├── values-security-prd.yaml
    └── values-observability.yaml
```

6.4. Resource Allocation

6.4.1. DataFetch API

Setting	Value
Replicas	1
CPU request / limit	100m / 250m
Memory request / limit	64Mi / 128Mi
Container port	8080
Service port	80 → 8080

6.4.2. Data Pipeline (CronJob)

Setting	Value
CPU request / limit	250m / 1000m
Memory request / limit	512Mi / 1Gi
Concurrency policy	Forbid (no overlapping runs)
Backoff limit	3
TTL after finished	600s
History (success/fail)	2 / 2

6.4.3. Database Migrations (PreSync Job)

Setting	Value
CPU request / limit	50m / 250m
Memory request / limit	128Mi / 256Mi
Backoff limit	0 (abort on failure)

6.5. Container Security

All containers run with the same hardened security context:

```yaml
securityContext:
  privileged: false
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  runAsNonRoot: true
  runAsUser: 1002
  capabilities:
    drop: [ALL]
podSecurityContext:
  fsGroup: 2000
  seccompProfile:
    type: RuntimeDefault
```

6.6. Health Probes (DataFetch API)

Probe	Path	Initial Delay	Period	Failure Threshold
Startup	`/monitoring/ping`	15s	10s	3
Readiness	`/monitoring/pong`	5s	10s	3
Liveness	`/monitoring/ping`	5s	10s	6

The readiness probe (`/monitoring/pong`) checks database connectivity. The DataFetch API is removed from the Service endpoints if the database becomes unreachable.

6.7. Secret Management

Secrets are managed by the External Secrets Operator, which synchronises credentials from HashiCorp Vault into Kubernetes Secrets every 5 minutes.

6.7.1. Vault Paths

All secrets are stored under the `instore/instore-orchestrator/` Vault path:

Secret Group	Vault Path	Contents
Database	`instore/instore-orchestrator/database`	DB host, port, username, password, database name, schema
Flyway	`instore/instore-orchestrator/database`	Flyway user, password, schemas, host, port, database
Connectors	`instore/instore-orchestrator/application`	Vusion Manager and VLink API credentials (URLs, keys)
TLS	`security/certificate/instore/instore-orchestrator`	Ingress TLS certificate and key

Each environment uses environment-specific property names within the same Vault paths (e.g., `instore_orchestrator_esl_pp_db_username` for preproduction).

6.7.2. Secret Injection

Secrets are injected as environment variables into containers:

DataFetch API: `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_SCHEMA`

Data Pipeline: Sink DB credentials (`SINK_POSTGRES_*`), state store DB credentials (`CONNECTOR_SYNC_STATE_STORE_POSTGRES_*`), Vusion Manager credentials (`CONNECTOR_VUSION_MANAGER_*`), VLink credentials (`CONNECTOR_VLINK_*`)

Flyway: `FLYWAY_USER`, `FLYWAY_PASSWORD`, `FLYWAY_SCHEMAS`, `FLYWAY_DB_HOST`, `FLYWAY_DB_PORT`, `FLYWAY_DB_NAME`

6.8. Network Policies

Network traffic is restricted using Kubernetes NetworkPolicies.

6.8.1. Ingress Rules

Source	Target Port	Purpose
Traefik ingress (`ingress-instore` namespace)	8080/TCP	External API access
Pods in same namespace	(any)	Inter-pod communication

6.8.2. Egress Rules

Destination	Target Port	Purpose
PostgreSQL nodes	10200/TCP (PP) / 5432/TCP (dev/prd)	Database connections
`kube-system/kube-dns`	53/UDP	DNS resolution
`observability/otlp-collector`	4317/TCP	OpenTelemetry traces

6.8.3. Database CIDRs (Preproduction)

Preproduction connects to three PostgreSQL replicas:

- `10.50.136.6/32` (port 10200)
- `10.50.136.7/32` (port 10200)
- `10.49.136.10/32` (port 10200)

Development and production use placeholder IPs (`10.0.0.1/32`) pending final infrastructure provisioning.

6.9. Ingress

The DataFetch API is exposed via a Traefik ingress with TLS termination:

Environment	Hostname
Development	`orchestrator-esl-datafetch-api-dev.corp.mc.pt`
Preproduction	`orchestrator-esl-datafetch-api-pp.corp.mc.pt`
Production	`orchestrator-esl-datafetch-api-prd.corp.mc.pt`

- Entrypoint: `websecure` (HTTPS only)

- TLS certificates are sourced from Vault via External Secrets Operator
- Ingress class: `traefik-instore`

6.10. Environment Differences

Setting	Development	Preproduction	Production
Namespace	`instore-esl-orchestrator-dev`	`instore-esl-orchestrator-pp`	`instore-esl-orchestrator-prd`
Pipeline schedule	`0 3 * * *` (3:00 AM)	`40 01 * * *` (1:40 AM)	`0 3 * * *` (3:00 AM)
Store filter	All stores	9 specific stores	All stores
DB port	5432	10200	5432
Image tags	Placeholder	Active	Placeholder
Secrets	Placeholder	Configured	Placeholder

Note: Development and production environments have placeholder values for image tags, secrets, and database CIDRs. These are populated when the environments are provisioned.

6.11. Monitoring and Alerting

Prometheus alert rules are deployed via the observability values file. Two alert groups are configured per environment:

6.11.1. Infrastructure Alerts

- Pod crash loops
- Pod not ready (> 15 min)
- Deployment rollout stuck
- Job completion failures
- Container waiting state

6.11.2. Application Alerts

Alert	Condition	Severity
HTTP 5xx error rate > 10%	15 min window	Critical
HTTP 5xx error rate 5–10%	15 min window	Warning
HTTP p95 latency > 3s	5 min window	Info
Average response time > 1s	5 min window	Info

Health check (`/monitoring/ping`) and metrics (`/metrics`) endpoints are excluded from alert calculations.

A global `maintenance` flag can be set to `on` to suppress all alerts during planned maintenance windows.

7. Operations & Maintenance

7.1. Configuration Changes

All runtime configuration for the ESL Orchestrator is managed through Git. Changes are made in the Kubernetes deployment repository and applied automatically via ArgoCD.

7.1.1. Repository

Configuration lives in the lac1041-instore-orchestrator-esl repository. The ArgoCD application tracks a specific branch per environment (e.g., `testing` for preproduction).

7.1.2. Git Workflow

1. Create a branch from the ArgoCD target revision:

```
git checkout testing
git pull origin testing
git checkout -b feature/<description>
```

2. Edit the relevant values file (e.g., `clusters/cluster-preproduction/values-pp.yaml`)
3. Commit using Conventional Commits:

```
git add clusters/cluster-preproduction/values-pp.yaml
git commit -m "chore: update esl datapipeline schedule"
git push origin feature/<description>
```

4. Open a pull request targeting the ArgoCD target revision branch
5. After merge, ArgoCD detects the change and syncs automatically

7.2. Common Operational Tasks

7.2.1. Changing the Pipeline Schedule

Edit the `schedule` field in the environment's values file under the data pipeline CronJob section:

```
- name: orchestrator-esl-datapipeline
  schedule: "0 6 * * *"   # Every day at 06:00 UTC
  timeZone: "UTC"
```

The schedule uses standard cron syntax. All times are in UTC.

Examples:

Schedule	Meaning
`"0 6 * * *"`	Every day at 06:00 UTC
`"0 */6 * * *"`	Every 6 hours
`"0 8 * * 1-5"`	Weekdays at 08:00 UTC
`"0 2 * * 0"`	Every Sunday at 02:00 UTC

> Note: `concurrencyPolicy: Forbid` is set, meaning a new run will be skipped if the previous one is still in progress.

7.2.2. Filtering Stores

Store filters restrict which stores the pipeline processes. Edit the connector settings in the values file:

Process specific stores:

```
connector:
  filter_stores:
    - "continente_pt.001234"
    - "bomdia_pt.009648"
```

Process all stores from specific retail chains:

```
connector:
  filter_stores: []
  filter_retail_chains:
    - "continente_pt"
```

Process all stores:

```
connector:
  filter_stores: []
```

> Priority: If `filter_stores` has entries, `filter_retail_chains` is ignored.

7.2.3. Running an Ad-Hoc Pipeline Execution

To trigger the pipeline outside of its schedule, create a Job from the CronJob:

```
kubectl create job --from=cronjob/orchestrator-esl-datapipeline \
  manual-run-$(date +%s) \
  -n instore-esl-orchestrator-pp
```

Monitor the job:

```
kubectl logs -f job/manual-run-<timestamp> -n instore-esl-orchestrator-pp
```

7.2.4. Checking Sync State

Query the `sync_state` table to see recent pipeline runs:

```
SELECT pipeline_name, sync_status, stores_processed,
       products_processed, labels_processed, access_points_processed,
       duration, started_at, finished_at, error_message
FROM esl.sync_state
ORDER BY finished_at DESC
LIMIT 10;
```

Query per-store sync history:

```
SELECT store_id, sync_status, products_processed,
       labels_processed, access_points_processed,
       synced_at, error_message
FROM esl.store_sync_state
WHERE pipeline_name = 'esl-orchestrator-pp'
ORDER BY synced_at DESC
LIMIT 20;
```

7.2.5. Viewing Migration Status

Check Flyway migration history from the migration job logs:

```
kubectl logs job/orchestrator-esl-datapipeline-db-migrations \
  -n instore-esl-orchestrator-pp
```

Or query the Flyway history table directly:

```
SELECT version, description, installed_on, success
FROM flyway_schema_history
ORDER BY installed_rank DESC
LIMIT 10;
```

7.2.6. Checking Failed Records

Query the error tracking table for records that failed during sync:

```
SELECT table_name, record_id, sync_mode,
       error_type, error_message, pg_error_code,
       created_at
FROM esl.records_with_errors
ORDER BY created_at DESC
LIMIT 20;
```

7.3. Troubleshooting

7.3.1. Pipeline Job Fails Immediately

Symptom: CronJob pod enters `Error` state and does not retry.

Possible causes:

1. Configuration error — invalid YAML in the ConfigMap. Check the pod logs:

   ```
   kubectl logs <pod-name> -n <namespace>
   ```

2. Secret not available — ExternalSecret has not synced. Check the ExternalSecret status:

   ```
   kubectl get externalsecret -n <namespace>
   ```

3. Database unreachable — network policy or database is down. Verify connectivity.

7.3.2. Migration Job Blocks Deployment

Symptom: ArgoCD sync stuck in PreSync phase.

Cause: The Flyway migration job failed (backoff limit is 0 — it does not retry).

Resolution:

1. Check the migration job logs for the SQL error
2. Fix the migration in the database repository
3. Push a new image tag
4. Update the image tag in the values file
5. ArgoCD will re-run the PreSync job with the new image

7.3.3. DataFetch API Returns 503

Symptom: Readiness probe fails, API removed from Service endpoints.

Cause: Database connectivity lost (readiness probe pings the database with a 2-second timeout).

Resolution:

1. Verify the database is accessible from the namespace
2. Check network policy egress rules
3. Check ExternalSecret for correct database credentials
4. Once connectivity is restored, the readiness probe will automatically pass and the pod will be added back to the Service endpoints

7.3.4. Incremental Sync Fetches Too Much Data

Symptom: Pipeline run takes longer than expected despite incremental mode.

Possible causes:

1. Large lookback window — the `lookback_window` setting adds a safety margin. If set too high (e.g., `24h`), it will re-fetch more data than necessary.
2. First run for new stores — newly added stores always do a full sync. Check the store filter to ensure only intended stores are included.
3. Previous run failed — if the last run for a store failed, the next run falls back to the last *successful* run's timestamp (or full sync if none exists).

7.3.5. Pipeline Processes Zero Records

Symptom: Sync state shows 0 records processed.

Possible causes:

1. Store filter mismatch — the configured `filter_stores` or `filter_retail_chains` don't match any stores in the Vusion API
2. API credentials invalid — check the connector secrets. The pipeline logs will show authentication errors.
3. No modified records — in incremental mode, if no records have been modified since the last sync, 0 is expected behaviour